MLOps MLflow Practice Report – Kubernetes Pipeline

MLOps Fundamentals - LTAT.02.038

Overview

In this practice, I built a local end-to-end MLOps pipeline using Kubernetes (Minikube), MLflow, and a FastAPI application. The goal was to move beyond simple containerization into a more realistic setup where model training, tracking, and serving are handled as separate, connected components. Yes, I used the help of ChatGPT, but it still took me 2 full days to gnaw through the what, how and why of things, especially the several issues due to running k8s locally on limited minikube.

The system includes:

- MLflow for experiment tracking and model storage
- a Kubernetes Job for model training
- a FastAPI app for inference and UI

Project Structure and Initial Setup

Before deploying anything to Kubernetes, I first structured the project so that training, serving, and deployment logic were clearly separated. This made the system easier to understand, containerize, and debug. It also gets closer to production-level setup than just any schoolwork I've done so far.

Project hierarchy

```
data/
src/
  __init__.py
  model_utils.py
  train.py
app/
  __init__.py
  app.py
  model_loader.py
  schemas.py
  templates/
    index.html
  static/
    style.css
docker/
  Dockerfile.mlflow
  Dockerfile.app
  Dockerfile.train
k8s/
  namespace.yaml
  pvc-mlflow.yaml
  mlflow-deployment.yaml
  mlflow-service.yaml
  app-deployment.yaml
  app-service.yaml
  train-job.yaml
  ingress.yaml
requirements-app.txt
requirements-train.txt
```

Why the files were separated

I split the project into smaller files so that each one had a clear role.

- `src/train.py` handles model training, evaluation, and MLflow logging.

- `src/model_utils.py` contains reusable ML logic such as data preparation, feature engineering, model building, and evaluation.
- `app/app.py` contains the FastAPI application and endpoints.
- `app/model_loader.py` contains the logic for resolving and loading the best model from MLflow.
- `app/schemas.py` defines request and response schemas for the API.
- `templates/` and `static/` keep the HTML and CSS separate from Python code, which keeps the app cleaner.

This separation helped because training and serving are different tasks and later run in different Kubernetes objects:

- training → Job
- app → Deployment

Python packages and `__init__.py`

I used `__init__.py` files to mark `src/` and `app/` as Python packages. They can be empty, but they make imports more reliable, especially in Docker and Kubernetes.

For example, imports such as:

```
from src.model_utils import build_model
from app.model_loader import load_model_from_mlflow
```

work more cleanly when the folders are treated as packages.

Why different requirements files were used

I used separate dependency files:

- `requirements-train.txt`
- `requirements-app.txt`

This was done because training and serving do not need exactly the same dependencies.

For example:

- training needs libraries such as pandas, pyarrow, scikit-learn, and MLflow
- the app needs additionally FastAPI, Uvicorn, Jinja2, and MLflow

In production, using separate files keeps Docker images smaller and clearer, and reflects the fact that each container has a different job.

Steps Performed

0. Local Development and Testing

I first developed and tested the application locally.

Built Docker images for:

- MLflow
- training script
- FastAPI app

Verified that:

- training works
- MLflow logs runs correctly
- app can load and use the model
- dockerisation works

This step helped ensure everything works before deploying to Kubernetes.

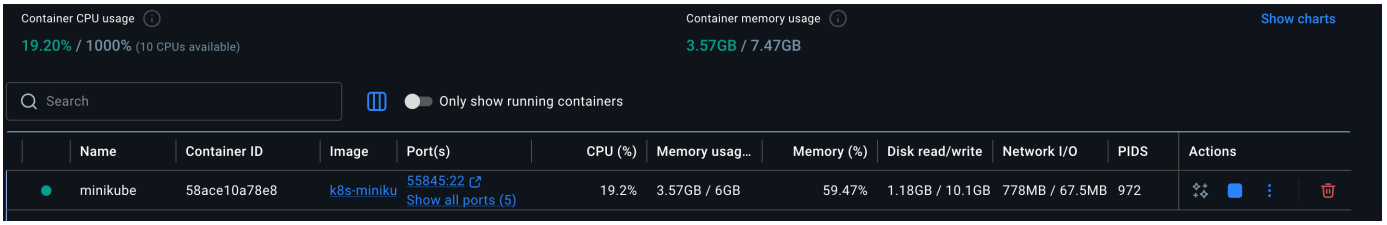
1. Kubernetes Setup

I thereafter configured Minikube (already installed during practice), then created a namespace (`mlops`):

```
kubectl apply -f k8s/namespace.yaml
```

I also learned the most important Kubernetes commands:

- `kubectl apply`
- `kubectl get pods`
- `kubectl logs`
- and most imporantly: `kubectl rollout restart`



Why this step was needed:
The namespace was created to keep all project resources grouped together under `mlops` instead of mixing them into the default namespace. This makes the system easier to inspect and clean up. Learning the basic `kubectl` commands was necessary because Kubernetes debugging relies heavily on checking pod states and logs.

---

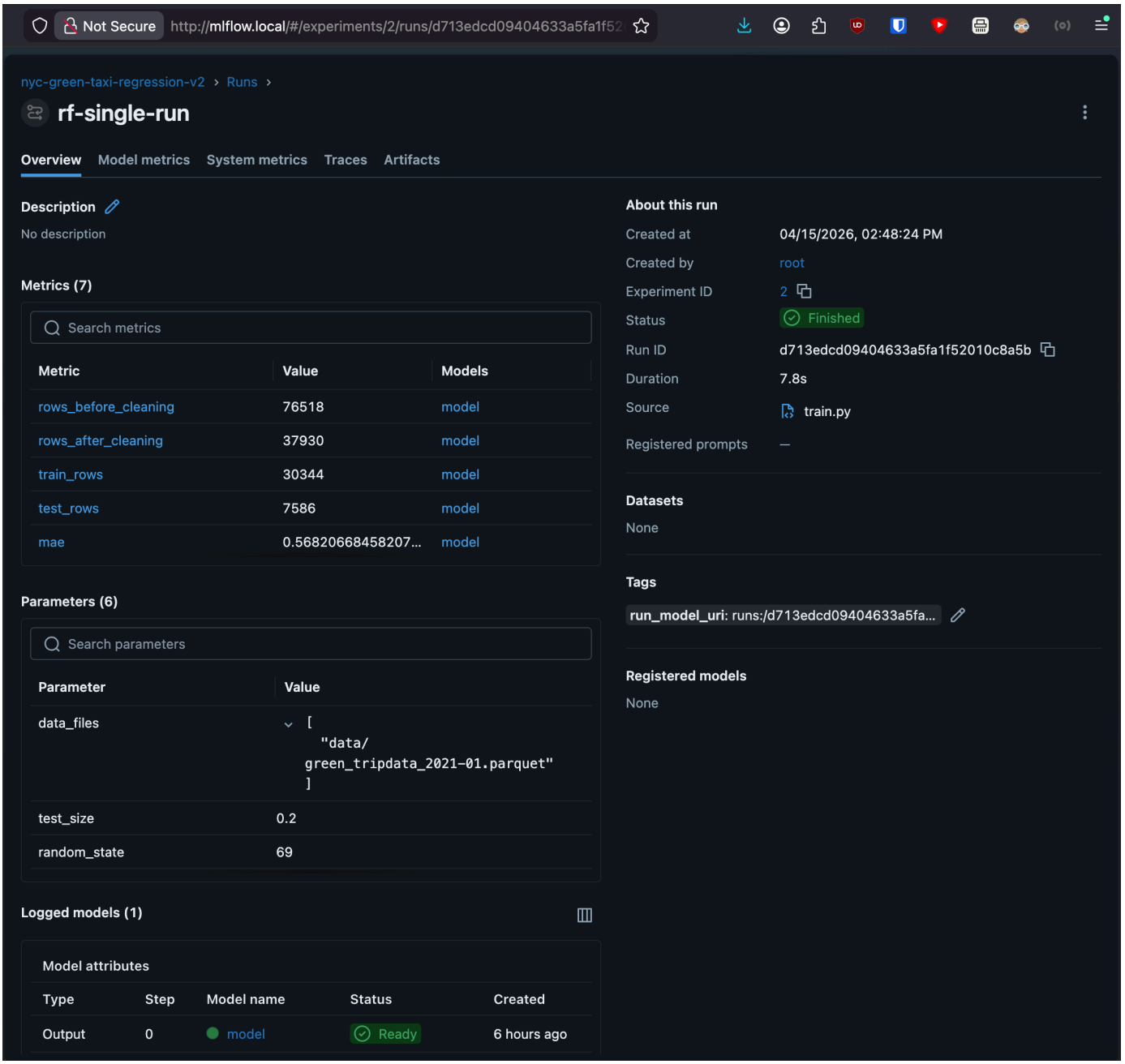
2. MLflow Deployment

I built a Docker image for MLflow and deployed it using:

- Deployment
- Service
- PersistentVolumeClaim (PVC)

MLflow was made accessible internally at:

- `http://mlflow-service:5001`



Why this step was needed:
MLflow is the central tracking component of the pipeline. The training job needs it to log experiments and models, and the app needs it to locate the best model. It had to be deployed first because both the training job and the app depend on it.

---

3. Training Pipeline

I created a `train.py` script that:

- loads parquet taxi data
- engineers features
- trains a `RandomForestRegressor`
- evaluates performance
- logs metrics and model to MLflow

I also saved the model URI into a run tag:

```
run_model_uri = f"runs:/{run_id}/model"
mlflow.set_tag("run_model_uri", run_model_uri)
```

The training was then executed through:

```
kubectl apply -f k8s/train-job.yaml
```

```
(.venv) Tarvos-MacBook-Pro:08_k8s woird$ kubectl get pods -n mlops -w
NAME                          READY   STATUS      RESTARTS   AGE
mlflow-8447c5df55-49d2j       1/1     Running     0          4h7m
taxi-app-65dd9484bb-48c4d     1/1     Running     0          4h54m
taxi-train-kj4lw              0/1     Completed   0          6h12m
```

Why this step was needed:
Training had to run inside Kubernetes to demonstrate a realistic pipeline rather than only local experimentation. By using a Job, the model training becomes a one-time task that starts, finishes, and exits cleanly, which is the correct Kubernetes pattern for this kind of workload.

---

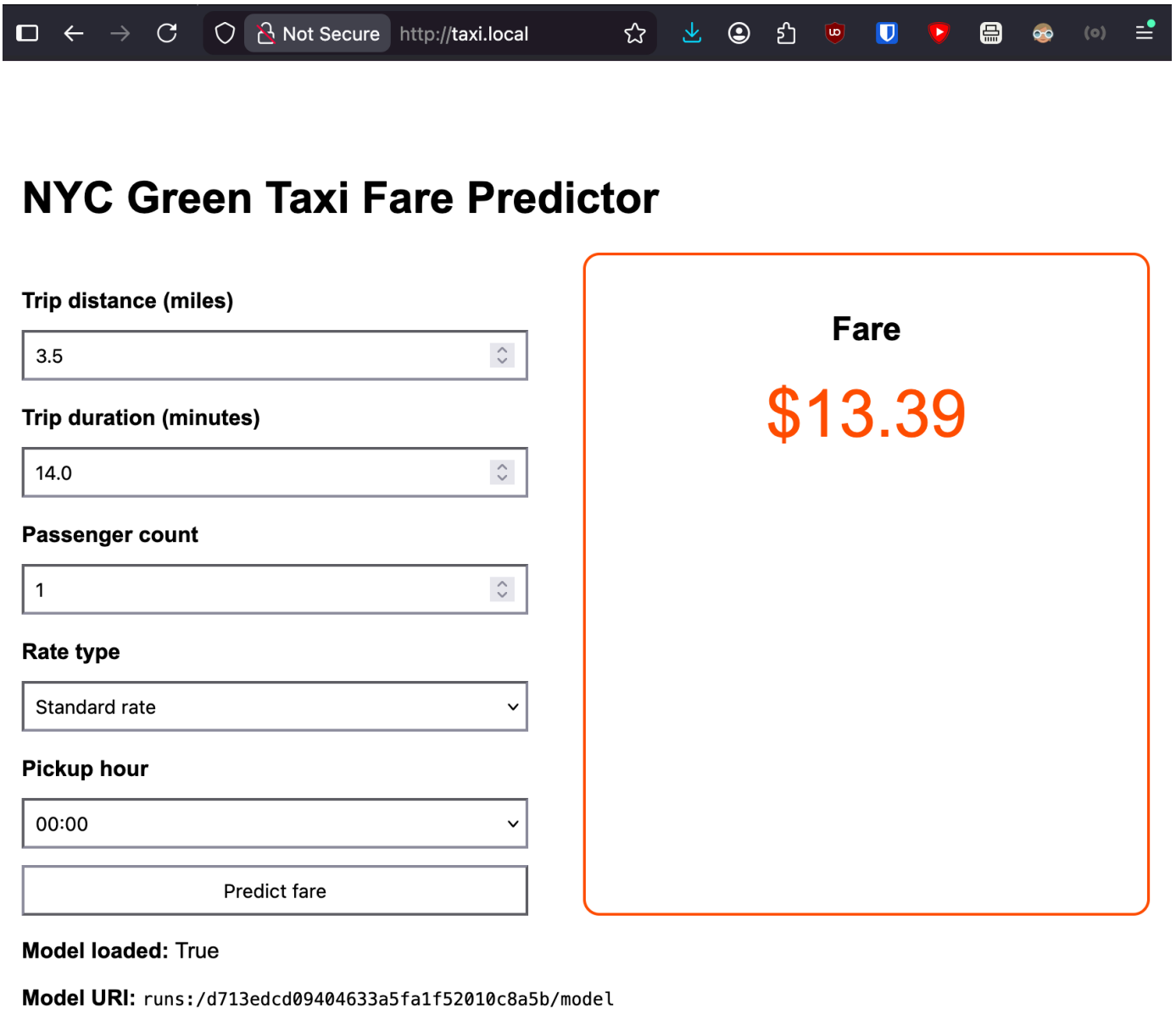
4. FastAPI Application

I built a FastAPI prediction app and added these endpoints:

- `/` for UI
- `/predict`
- `/health`
- `/model-info`

The app loads the best model at startup by:

- querying MLflow runs
- sorting them by RMSE
- loading the model from the best run



Why this step was needed:
The app represents the serving layer of the pipeline. Without it, the project would only show training and experiment tracking, but not model deployment and real use. The UI was also useful for screenshots and for demonstrating the prediction workflow more clearly than raw API calls alone.

---

5. Connecting Components & ingress

I connected MLflow and the app using Kubernetes Services:

- `mlflow-service`
- `taxi-app-service`

This allowed internal communication through service names rather than IPs.

I then exposed services via clean URLs:

- `http://taxi.local` → FastAPI app
- `http://mlflow.local` → MLflow UI
- Enabled Minikube ingress addon
- Created `Ingress` resource routing:
  - `taxi.local` → `taxi-app-service:8000`
  - `mlflow.local` → `mlflow-service:5001`
- Very imporantly, ran:

```
minikube tunnel
```

```
(.venv) Tarvos-MacBook-Pro:08_k8s woird$ kubectl get svc -n ingress-nginx
NAME                                 TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)                      AGE
ingress-nginx-controller             NodePort    10.111.39.16    <none>        80:31837/TCP,443:31702/TCP   37m
ingress-nginx-controller-admission   ClusterIP   10.106.40.165   <none>        443/TCP                      37m
```

Why this step was needed:
Pods in Kubernetes are temporary and their IPs can change. Services provide stable internal endpoints, which makes communication reliable. This is why the app can use `http://mlflow-service:5001` instead of hardcoded addresses.

Challenges and Fixes

Challenge	Solution	Why it occurred
OOMKilled training job	Reduced dataset size and model size	Minikube had limited memory, and the training workload was too heavy at first
Docker images not found in Kubernetes	Used `eval $(minikube docker-env)` before building	Docker images were built in the local Docker context instead of Minikube's Docker environment
Model not loading in app	Added shared PVC mounted at `/mlflow`	MLflow and app pods originally had isolated filesystems, so the model files were not visible across pods
MLflow URI issues	Switched to `runs:/<run_id>/model`	MLflow v3 logged model URIs were not working reliably in this setup
App showed `model_uri = None`	Restarted the app deployment after training	The model was loaded only once during app startup, so it did not automatically refresh
`kubectl port-forward` failing	Used `minikube service --url` instead	Local Minikube/Docker Desktop networking was unstable
MLflow host validation error	Added allowed hosts and CORS config	MLflow blocked internal requests for security reasons
Job could not be updated after changes	Deleted and recreated the Job	Kubernetes Jobs are not meant to be updated in place like Deployments

---

Learnings and Understanding

This exercise helped me understand how real-world ML systems are structured and deployed.

First, Kubernetes provides a way to manage separate components such as training, tracking, and serving independently. Each component runs in its own container, but still communicates through Services. This improves modularity and makes the system easier to reason about.

Second, MLflow integration is not just about logging metrics. Proper handling of model artifacts and model URIs is essential. I learned that storing and retrieving models requires understanding where the files are actually located and how different services access them. It took me quite a while, even with help from ChatGPT!

Third, resource constraints matter a lot, even with a M1 Pro Macbook. Running ML workloads locally on Minikube requires adapting the dataset size and model complexity to available memory. This is a practical engineering concern, not just an optimization detail.

Fourth, debugging distributed systems is different from debugging local code. Many of the problems I faced were not caused by Python itself, but by the surrounding environment:

- Docker context mismatches
- Kubernetes networking
- storage visibility
- memory limits

Finally, I saw how MLOps tools fit into one workflow:

- training → tracking → serving

This made the project much more realistic than only training a model locally.

Running the Full Pipeline

To train and use the best model, I now do the following:

1. Run training

```
kubectl delete job -n mlops taxi-train --ignore-not-found
kubectl apply -f k8s/train-job.yaml
```

2. Wait for completion

```
kubectl get jobs -n mlops
```

3. Restart app to load best model

```
kubectl rollout restart deployment/taxi-app -n mlops
```

4. App logic

The app then:

- queries MLflow
- sorts runs by RMSE in ascending order
- selects the best run
- loads model from:

```
runs:/<run_id>/model
```

This means the app always picks the currently best-performing model according to the tracking data.

Grid Search Note

Grid search has been implemented in `app.py`, but it is not currently used in the Kubernetes training pipeline. A future improvement would be to add it to the training Job so that multiple hyperparameter combinations could be tested directly in the cluster before selecting the best model.

Conclusion

I successfully built a working MLOps pipeline using Kubernetes, MLflow, and FastAPI. While several issues arose related to environment setup, storage handling, and resource limits, resolving them gave me a much better understanding of how real-world ML systems operate and interact.